Verifiable Chain-of-Custody for Digital Evidence Files

A Standalone Hash-Chain and Merkle-Tree Approach Without Distributed Consensus

Chilakapati Srijaya Chakradhar
Dept. of Computer Science and Engineering
KL University (Hyderabad)
Hyderabad, India
jaya.chakri2005@gmail.com

Gogineni Nikhil Sai
Dept. of Computer Science and Engineering
KL University (Hyderabad)
Hyderabad, India

Pusunuru Preetham
Dept. of Computer Science and Engineering
KL University (Hyderabad)
Hyderabad, India

Dr. Archana Kalidindi (Supervisor)
Dept. of Computer Science and Engineering
KL University (Hyderabad)
Hyderabad, India

Abstract—Digital forensic investigations live and die on whether an evidence artifact a disk image, a memory dump, a browser cache export can be shown to be exactly what it was at the moment of collection. Conventional evidence-handling workflows do not actually provide that guarantee; they provide a paper trail around it. Once a file is stored or handed from one analyst to the next, nothing in a logbook or an access-controlled folder can prove, after the fact, that the file itself is unaltered, or that every entry in its custody history is genuine. This paper presents a chain-of-custody system built from first principles that closes that gap without adopting distributed-ledger infrastructure. Each evidence file is hashed at intake with SHA-256 and recorded as a leaf in a Merkle tree, so that any single file's membership in the registered evidence set can be checked in logarithmic time. Custody actions transfer, access, annotation are appended to a hash-linked, ECDSA-signed log, and a separate, independently implemented verifier recomputes every hash and signature from the raw artifacts rather than trusting the tool that wrote them. Beyond the core design, we extended the system in two directions this cycle: an encrypted-at-rest SQLite backend (AES-256-GCM) for evidence and custody metadata, and support for registering an entire evidence folder not just a single file as one batch, with the folder's own Merkle sub-tree root folded into the main evidence tree as a nested, two-hop proof. A synthetic tamper-simulation harness confirms that all four tested single-point corruption scenarios are detected and correctly localized to the offending log entry, and a 110-test suite exercises every module, including the folder-batch and nested-proof additions. The central argument is not that blockchain's tamper-evidence properties are undesirable for forensic custody they are exactly what this domain needs but that they do not require blockchain's consensus layer, because a single accredited forensic lab acting as sole custodian is not the multi-party, mutually-distrusting setting consensus exists to serve.

Keywords—*chain of custody, digital forensics, Merkle tree, hash chain, ECDSA, tamper-evident logging, digital evidence integrity*

I. INTRODUCTION

Chain of custody, in the forensic sense, is the record of who collected, handled, transferred, or examined a piece of evidence, and when. Its integrity is not a nice-to-have; it is often the difference between evidence that is admissible and evidence that is not. If an investigator cannot demonstrate that a file's provenance and unaltered state hold up under scrutiny, the technical content of that file however damning carries little evidentiary weight.

Most digital forensics labs still manage this with procedural controls: a logbook, a locked evidence room, a policy document specifying who is authorized to touch what. These controls are not worthless, but they share two structural weaknesses. First, nothing in them lets an outside party independently confirm, after the fact, that a stored file or its custody record has not been quietly altered the guarantee rests entirely on trusting the people and processes that maintained it. Second, when tampering is eventually suspected, a logbook offers no way to pinpoint exactly where the chain of trust broke; at best it prompts a manual audit of every entry.

A body of recent work, reviewed in Section III, has proposed solving this with blockchain usually a private Ethereum deployment or a permissioned Hyperledger network. These systems do gain genuine tamper-evidence from blockchain's consensus layer, but they also inherit its cost: standing up and operating a distributed ledger, running peer nodes, maintaining consensus infrastructure. That cost buys something specific agreement among multiple, mutually distrusting parties on one shared history and a single accredited forensic lab acting as sole custodian of its own evidence does not have that problem. There is one system of record and no competing party trying to rewrite history in parallel.

This paper's contribution is to take blockchain apart into the pieces that actually do the tamper-evidence work content hashing, hash-linking of records, Merkle-tree inclusion proofs, and asymmetric signatures and build a working system from exactly those pieces, deliberately leaving out the consensus and distributed-ledger layer. The result is a small, dependency-light Python tool: evidence files are hashed and indexed in a Merkle tree, custody actions are hash-chained

and signed with ECDSA, and a standalone verifier implemented independently enough that it shares no code path with the logging tool recomputes every check from the raw log rather than trusting the logger's own report of its state.

II. MOTIVATION

Three observations from the literature survey (Section III) shaped this project's direction. The first is unremarkable but worth stating plainly: manual, logbook-based custody processes are slow, error-prone, and not intrinsically auditable a point argued at length in recent review literature (Patil et al., 2024). The second is that the reviewed blockchain-based chain-of-custody systems (Bonomi et al., 2018; Lone & Mir, 2019; Ahmad et al., 2020) do achieve real cryptographic tamper-evidence, and that part of their design is worth keeping but every one of them pays for it with distributed-ledger infrastructure, whether that is a private Ethereum network, a Hyperledger Composer deployment, or smart-contract and wallet machinery, regardless of whether the deployment actually has multiple distrusting parties who need that infrastructure. The third observation comes from outside the forensics literature entirely: non-blockchain tamper-evident logging research (Crosby & Wallach, 2009; Yagiz et al., 2026) shows that Merkle-tree-based tamper evidence is practical and fast without any distributed ledger at all but that line of work targets generic log integrity, things like IoT telemetry streams, not forensic evidence with investigator-attributable, digitally signed custody actions.

Put together, these three observations describe a gap rather than a contradiction. The non-blockchain tamper-evident-logging literature has the right cost profile but is missing actor attribution it can tell you a log wasn't altered, but not who is accountable for a given entry. The blockchain-CoC literature has actor attribution (via signatures bound to on-chain identities) but pays a consensus tax it does not need for a single-custodian deployment. This project sits at that intersection: it borrows the tamper-evidence techniques the non-blockchain literature already validated, adds the digital-signature actor-attribution property the blockchain-CoC literature correctly identifies as necessary, and drops the consensus layer that literature carries for a threat model multiple mutually distrusting custodians that a single accredited lab simply does not have.

III. RELATED WORK

Ten sources were reviewed for this project; each is summarized below with its specific relevance. Fuller annotated summaries live in the project's companion research-summary document.

- **Bonomi, Casini, & Ciccotelli (2018)** propose B-CoC, a private-Ethereum blockchain chain-of-custody system with a working prototype and reported latency/storage benchmarks. This is arguably the closest prior art in spirit to this project, though it remains dependent on operating an actual blockchain network to get its guarantees.
- **Yagiz, Horasan, & Yurttakal (2026)** present a Merkle-tree pipeline with adaptive chunking for tamper-evident IoT log verification, reporting throughput above 130,000 entries per second with no distributed ledger involved at all. Technically this is the closest match to this project's non-blockchain approach, though the target domain is generic telemetry rather than forensic evidence.
- **Crosby & Wallach (2009)** is the foundational paper on Merkle-tree-chained "history trees" for tamper-evident logs, with O(log n) inclusion and consistency proofs. Its algorithms directly inform this project's own Merkle-tree module.
- **Lone & Mir (2019)** propose Forensic-chain, a permissioned Hyperledger Composer framework using hash pointers to build an append-only evidence trail again, this requires deploying and maintaining a distributed ledger.
- **Ahmad, Khanji, Iqbal, & Kamoun (2020)** extend blockchain-based chain of custody to physical evidence via smart-lock integration on a private Ethereum network, adding an IoT hardware dependency on top of the ledger dependency.
- **Patil, Kohli, Puri, & Puri (2024)** is a review paper arguing for blockchain's potential to replace manual, logbook-based custody records. It frames much of this project's own motivation without prescribing blockchain as the only viable technical route.
- **Al-Khateeb, Epiphaniou, & Daly (2019)** argue, at a conceptual level, that evidentiary integrity should be demonstrable through a verifiable, challengeable record framing the legal-admissibility case for cryptographic verifiability in general, independent of any particular implementation.
- **Prakash & Sadawarti (2022)** propose a secure digital-evidence framework combining RC4 encryption with fuzzy hash segmentation, benchmarked via Hyperledger Caliper. Its emphasis encrypting evidence content is different from this project's focus on tamper-evident event chaining.
- **Miller & Singh (2024)** explore Ethereum smart contracts and wallet-based signing to record custody events as signed, replay-resistant on-chain transactions. This directly supports one of this project's central claims that signed events, not hashing alone, are what strong custody guarantees require though it is realized through smart-contract and wallet infrastructure rather than a standalone signer.
- **Sakshi, Malik, & Sharma (2023)** apply a standard block-header design Merkle tree, previous-block hash, timestamp to preserve IoT-sourced multimedia evidence, which confirms that this project's own block-header-style pattern is a recognized, established structure in the forensic literature rather than a novel invention.

One citation note is worth recording here for transparency: during an internal citation-integrity pass, reference 9 (Miller & Singh) was found to be mis-recorded in an earlier literature summary as "Chain of Custody and Evidence Integrity Verification Using Smart Contracts" (2021). The verified title and venue are "Chain of Custody and Evidence Integrity

Verification Using Blockchain Technology," ICCWS 2024 corrected in the reference list (Section IX).

IV. METHODOLOGY AND SYSTEM ARCHITECTURE

The system decomposes "blockchain" into four cryptographic ingredients content hashing, hash-linking, Merkle trees, and asymmetric signatures and implements each one directly, while deliberately leaving out the consensus and distributed-ledger layer. Table I summarizes which ingredients survive the cut and why.

Ingredient	Kept?	Role in this system
Cryptographic hashing (SHA-256)	Yes	Content-addresses every evidence file; the basis for detecting any byte-level change
Hash-linking of records	Yes	Custody entries reference the previous entry's hash, so altering entry N invalidates every later entry
Merkle tree	Yes	Confirms one file's inclusion in the evidence set without re-hashing every other file
Asymmetric signatures (ECDSA)	Yes	Binds a specific actor to a specific custody action; forging an entry requires the private key, not just hash knowledge
Distributed consensus / mining	No	Solves multi-party disagreement, absent under a single-custodian threat model
Public / permissionless ledger	No	Evidence confidentiality is a requirement here, not a non-goal

Table I. Blockchain ingredients kept vs. omitted, and the role of each in this system.

A. System Architecture

The pipeline runs evidence through seven stages: ingestion, SHA-256 hashing, Merkle-tree construction, hash-chain custody logging, ECDSA signing, optional encrypted storage, and independent verification. Each stage lives in its own module hashing.py, merkle.py, hash_chain.py, ecdsa_signer.py, storage.py with no circular dependencies between them, which is what lets the verifier depend only on the two primitives it actually needs (hashing and signature verification) without ever importing the logging tool's own logic.

B. Hash-Chain Custody Log

Each custody event is a record index, evidence_id, actor, action, timestamp, prev_hash. The entry_hash for record i is SHA-256 over a canonical, deterministic serialization of those fields concatenated with prev_hash, where prev_hash is simply the entry_hash of record i−1 (or a fixed genesis value for the very first record). This is the identical hash-linking idea used in blockchain block headers, applied here to a strictly linear, single-writer log instead of a multi-node ledger the link still makes altering any past entry detectable, because it does not need a network of peers to agree on which chain is canonical, since there is only ever one chain.

C. Merkle Tree over Evidence

Every registered evidence file's SHA-256 digest becomes a leaf in a Merkle tree, with odd leaf counts handled by duplicating the final node, following the usual convention. This gives an inclusion proof of size O(log n) that certifies a specific file's digest is part of the currently committed evidence set, without needing to disclose or re-hash every other file in that set.

D. ECDSA Signing

Every custody entry's hash is signed with an ECDSA (SECP256k1) private key held by the custodian, and the signature travels alongside the entry. This is the actor-attribution property that Section III identifies (via Miller & Singh, 2024) as necessary for a custody guarantee stronger than hashing alone implemented here as a standalone signer rather than smart-contract or wallet infrastructure, since there is no on-chain identity system to plug into.

E. Threat Model

In scope: a single-custodian forensic lab with one system of record; insider or after-the-fact tampering with a past custody entry or an evidence file, which the hash chain and Merkle tree make detectable not preventable at write time by an independent verifier; and data-at-rest confidentiality via the optional AES-256-GCM-encrypted SQLite backend, where corrupted ciphertext simply fails to decrypt rather than silently returning corrupted plaintext.

Explicitly out of scope: Byzantine fault tolerance or multi-party consensus, because there is no set of mutually distrusting nodes that need to agree on a single canonical history in this deployment target; real-time prevention of insider tampering this system detects modification after the fact, it does not stop someone who already has write access to the log file from editing it before the next verification run; and recovery from private-key compromise, which would require multi-signature or threshold signing, or HSM-backed key storage noted here as an explicit boundary of the current design rather than a solved problem (Section VIII).

V. IMPLEMENTATION

The system is written in Python 3.11 with a deliberately thin dependency footprint: the ecdsa package for SECP256k1 signing and verification, the cryptography package for AES-256-GCM, and otherwise only the standard library hashlib, sqlite3, argparse, json. Code-level detail is kept out of this section on purpose; the repository is the source of truth. What follows is structure and interfaces.

A. Module Structure

Module	Responsibility
vcoc/hashing.py	Streaming SHA-256 for files (fixed-size chunks) and byte strings
vcoc/merkle.py	Merkle tree construction, inclusion-proof generation and verification, plus two-hop nested proofs for folder-batch evidence (Section V-E)
vcoc/hash_chain.py	Append-only, hash-linked, ECDSA-signed custody log with an independent verify() method
vcoc/ecdsa_signer.py	SECP256k1 keypair generation, signing, verification
vcoc/storage.py	SQLite backend; every row is an AES-256-GCM ciphertext with its primary key bound in as AEAD associated data
vcoc/interactive/	prompt_toolkit-based interactive shell menu-driven access to every CLI command, live evidence search,

Module	Responsibility
	filesystem-autofill path prompts (Section V-D)
cli.py	argparse CLI: init-keys, add-evidence (file or folder), log-event, verify-chain, merkle-root, merkle-proof, shell
verifier.py	Standalone verifier process; re-implements every hash, signature, and Merkle check with no import of cli.py, hash_chain.py, or merkle.py
scripts/tamper_simulation.py	Experiment harness: builds a clean chain, applies four single-point corruptions, confirms detection

Table II. Module structure and responsibilities.

B. Independence of the Verifier

One design requirement carried all the way through implementation, not just stated as an intention, is that verifier.py must not share a trust root with the tool that produced the log. It runs as a separate Python process, reads only the artifacts an external auditor would actually have access to the JSON log file and the public key and re-derives every hash and signature check directly from stdlib hashlib and the ecdsa package, rather than importing vcoc.hash_chain or vcoc.merkle. The practical consequence is that a bug, or a deliberate backdoor, in the logging CLI cannot cause the verifier to falsely certify a tampered log as authentic the two code paths simply do not intersect.

C. Encrypted Storage Extension

The optional SQLite backend (vcoc/storage.py) stores evidence metadata and custody events as AES-256-GCM ciphertext blobs, each with a 96-bit random nonce prepended and its authentication tag included. Each row's primary key is bound in as AEAD associated data, so copying one row's ciphertext onto a different row's key simply fails to decrypt this defends specifically against row-swap tampering, not just per-field corruption of a single row in isolation.

D. Interactive Shell

The original interface to this system was a set of one-shot CLI subcommands python cli.py add-evidence --file ... --evidence-id ..., and so on for every operation. This works, but it requires remembering and retyping exact evidence IDs and flag names for every action, which is awkward in an interactive investigation session. We added a full-screen, menu-driven interactive shell (python cli.py shell), built with prompt_toolkit, that wraps every existing one-shot subcommand behind arrow-key navigation and live search: typing a partial evidence ID or filename filters the evidence set on every keystroke, and file-path prompts show filesystem autofill instead of a blind text box. The shell is strictly additive every cmd_* function it calls is the same, unmodified function the one-shot CLI calls, so the two interfaces are provably behavior-identical rather than two parallel implementations that could drift apart.

E. Folder-Batch Registration and Nested Merkle Proofs

Real forensic evidence is rarely a single file. A seized disk image, once extracted, is a directory tree of thousands of files; registering each one by hand through a single-file add-evidence call does not scale to how evidence actually arrives. We extended add-evidence both the one-shot subcommand and the interactive shell's corresponding menu action to accept a folder path instead of a file. Pointing it at a folder walks every file recursively (skipping symlinks and special files, never following a symlink into a loop), hashes each one, and registers it as its own Evidence record under an id scheme of the form {folder_id}/{relative_path} so per-file operations such as check-evidence and search continue to work exactly as they do for a solo file. The folder's own file hashes are then built into a Merkle sub-tree, and that sub-tree's root becomes a single leaf in the main evidence tree, alongside any ordinary single-file leaves.

This makes the inclusion proof for a file inside a registered folder a genuine two-hop, nested proof: one segment proves the file's membership in its folder's own sub-tree, and a second segment proves that sub-tree's root is included in the main tree. A file registered on its own, outside any folder, is handled by the same NestedProof structure with its second segment empty and its sub-tree root equal to the main root directly the single-file case is a literal degenerate instance of the two-hop format rather than a second code path maintained in parallel. Because the standalone verifier deliberately shares no code with the logging tool (Section V-B), it needed its own independent implementation of this same two-hop verification logic; we added that in lockstep and confirmed, with a dedicated test, that the verifier's independently recomputed root over a mixed set of solo files and one folder batch agrees exactly with the root the logging tool itself reports.

F. Testing

The pytest suite currently comprises 110 tests (109 passing, 1 environment-gated symlink test skipped on platforms without symlink permission) covering: SHA-256 correctness against the stdlib reference and single-byte tamper detection; Merkle proof correctness across parametrized leaf counts including odd counts, plus the nested two-hop proof format and its degenerate single-file case; hash-chain tamper detection for payload, prev_hash, and signature corruption; ECDSA sign/verify round trips and failure modes; encrypted-storage round trips and failure modes including wrong-key, tampered-ciphertext, wrong-associated-data, and row-swap tamper scenarios; folder-batch registration end to end, including deterministic re-registration (the same folder hashed twice produces the same sub-tree root) and symlink handling; the interactive shell's menu dispatch, live search, and filesystem-autofill prompts via headless key-sequence tests against a simulated terminal; and subprocess-level tests that invoke the standalone verifier and the tamper-simulation harness as genuinely separate external processes, to confirm CLI-level behavior rather than only library-level behavior.

VI. EXPERIMENTAL SETUP AND RESULTS

Five experiment classes were defined at the planning stage: (1) correctness on clean data, (2) tamper detection, (3) performance and scale versus evidence-file count, (4) storage overhead of plaintext versus AES-GCM-at-rest, and (5) extension-specific tests. All evidence used across these experiments is synthetically generated rather than drawn from real forensic case data a deliberate choice that avoids any chain-of-custody or privacy concern around test data while still exercising the system's actual code paths exactly as they would run against real evidence. Results for (1)–(4)

below are reproduced from actual runs against this repository's implementation; raw JSON output and rendered charts are kept alongside the project documentation. Extension-specific timestamp tests (5) do not apply this cycle, since RFC 3161 / OpenTimestamps anchoring was considered and not selected as a scope extension (Section VIII).

A. Correctness on Clean Data

A freshly built custody chain and its corresponding Merkle inclusion proofs were verified with the standalone verifier. Result: pass verifier.py reports "OK: chain of N entries verified independently" and "OK: Merkle inclusion proof … verified against root …" for clean, untampered input, confirming that the independent implementation agrees with the logging tool's own hash and signature computation.

B. Tamper Detection

Four independent single-point corruption scenarios were run via scripts/tamper_simulation.py against a clean five-entry chain, each applied to a freshly rebuilt clean chain so scenarios do not compound with one another. Table III reports the results of the most recent run.

Scenario	Corruption applied	Tgt.	Det.?	Rep.	Reason
flip_actor_field	Change actor name on entry 2	2	Yes	2	entry_hash mismatch
flip_action_field	Change action on entry 1	1	Yes	1	entry_hash mismatch
corrupt_signature	Flip one hex char in entry 3's signature	3	Yes	3	signature failed
break_prev_hash_link	Corrupt entry 4's prev_hash	4	Yes	4	prev_hash mismatch

Table III. Tamper-detection results across four single-point corruption scenarios.

All four scenarios 100% were correctly detected, and in every case the verifier reported the exact entry index at which tampering occurred, rather than merely flagging that the chain as a whole was invalid. This is the direct experimental support for the paper's central tamper-evidence claim: a single-custodian, non-consensus design can still make any retroactive alteration to the custody record detectable and localizable, down to the specific entry and the specific reason.

C. Performance / Scale

scripts/performance_scale.py generates N synthetic 64 KB evidence files (N = 10, 100, 1,000), then times SHA-256 hashing of all N files, Merkle-tree construction over the N digests, inclusion-proof generation for one leaf, and inclusion-proof verification. Table IV reports the results.

Files	Hashing (s)	Merkle build (s)	Proof gen (s)	Proof verify (s)	Proof steps
10	0.0037	0.000027	0.0000084	0.0000049	4
100	0.0404	0.000095	0.0000129	0.0000086	7
1,000	0.4286	0.000985	0.0000144	0.0000114	10

Table IV. Hashing, Merkle-build, and proof performance versus evidence-file count.

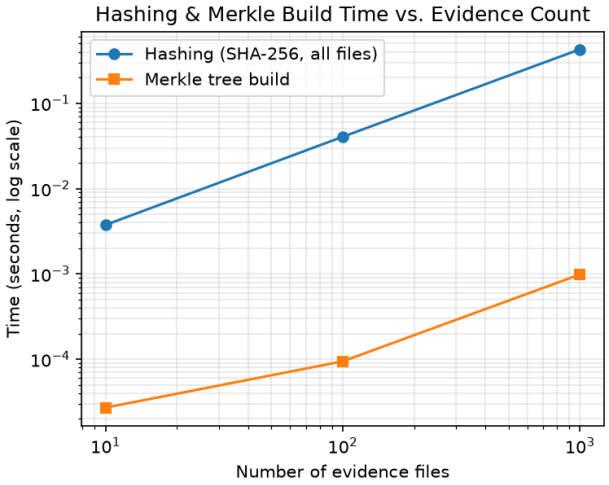


Fig. 1a. Hashing and Merkle-tree build time (log-log) versus evidence-file count.

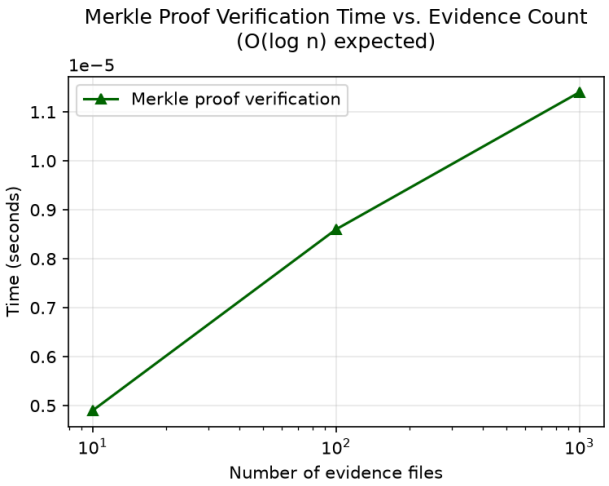


Fig. 1b. Merkle proof verification time versus evidence-file count.

Hashing time scales linearly with evidence-set size, as expected each file is streamed through SHA-256 independently at roughly 0.37–0.43 ms per 64 KB file regardless of N. Merkle-tree build time also scales linearly in the number of leaves but stays under 1 ms even at 1,000 files, negligible next to hashing cost. Proof size grows as $O(\log_2 n)$, exactly as predicted: 4, 7, and 10 steps at 10, 100, and 1,000 leaves respectively $\text{ceil}(\log_2 n)$ in each case. Most notably, proof-verification time stays in the low-microsecond range, 4.9 to 11.4 microseconds, across a hundredfold increase in evidence-set size only about a 2.3x increase in verification time for a 100x increase in scale, consistent with O(log n) behavior and confirming that an auditor can cheaply verify one file's inclusion without re-hashing the entire evidence set even as that set grows large.

D. Storage Overhead

scripts/storage_overhead.py builds N synthetic evidence records plus N genuinely hash-linked, ECDSA-signed custody events, then compares their size as plain JSON the plaintext baseline against the resulting file size of an EncryptedStore (SQLite plus AES-256-GCM) populated with the same records. Table V reports the results.

Records	Plaintext (B)	Encrypted (B)	Overhead
10	6,725	24,576	+265.4%
100	67,025	102,400	+52.8%
1,000	670,925	839,680	+25.2%

Table V. Plaintext versus AES-256-GCM-encrypted storage size, by record count.

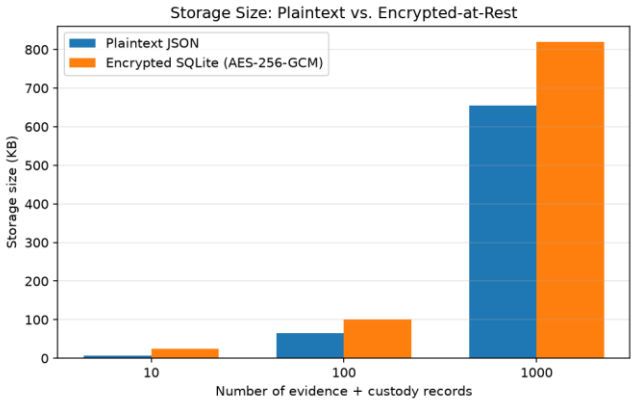


Fig. 2. Plaintext JSON size versus encrypted (AES-256-GCM) SQLite size, by record count.

AES-256-GCM itself only adds a 12-byte nonce and a 16-byte authentication tag per record 28 bytes of genuine cryptographic overhead on a roughly 670-byte plaintext record, about 4%. The measured overhead is far larger than that at small scale, +265% at 10 records, because SQLite allocates storage in fixed-size pages; the observed minimum file size is 24,576 bytes regardless of how little data is actually stored. As record count grows, this fixed per-database overhead amortizes across more records, so the percentage overhead decreases with scale 265% to 53% to 25% even though the absolute overhead in bytes keeps growing. This is a genuinely counter-intuitive finding worth stating plainly: for this extension, storage overhead is a property of the storage engine's page-allocation strategy, not of the encryption scheme itself. Isolating the two effects comparing against an unencrypted SQLite baseline with the same page allocation but no AES-GCM is a natural refinement left for a future iteration.

E. Folder-Batch Registration and Nested-Proof Performance

To check whether the folder-batch extension (Section V-E) introduces a meaningful performance cost, we measured end-to-end folder registration time and nested-proof generation and verification time for folders of 10, 100, and 1,000 synthetic 64 KB files, using the same file sizes as the single-file scale experiment above for direct comparability. Table VI reports the results.

Files in folder	Registration (s)	Nested proof gen (s)	Nested proof verify (s)
10	0.0080	0.00048	0.000008
100	0.0628	0.00077	0.0000137
1,000	0.6337	0.00501	0.0000218

Table VI. Folder-batch registration and nested Merkle-proof performance versus folder size.

Registration time is dominated by per-file SHA-256 hashing, as it is in the single-file case, and scales linearly with folder size roughly 0.6–0.8 ms per file, consistent with the earlier hashing-only measurement once the additional per-file bookkeeping (building the Evidence record, appending to the folder's member list) is accounted for. Nested-proof verification stays in the same low-microsecond regime as flat, single-hop verification 8 to 22 microseconds across a hundredfold increase in folder size because the second hop of the proof only ever needs to walk the main tree's own O(log n) depth over the number of top-level evidence entries (files and folders combined), not the number of files inside any one folder. In every trial, the standalone verifier's independently recomputed root for the resulting mixed evidence set matched the root the logging tool itself reported, confirming that the two-hop proof format does not introduce any drift between the two independent implementations.

VII. DISCUSSION

The tamper-detection results in Section VI-B support this paper's central claim: hash-chaining, Merkle proofs, and digital signatures together give the same tamper-evidence property commonly attributed to blockchain, without needing blockchain's consensus layer to get it. This is consistent with and extends the non-blockchain tamper-evident-logging literature (Crosby & Wallach, 2009; Yagiz et al., 2026) into a forensic setting that additionally demands actor attribution via ECDSA signatures, something that literature's generic log-integrity focus does not address.

This design's boundaries are worth stating explicitly rather than leaving them implicit, and there are three of them. First, the system detects tampering after the fact; it does not prevent a custodian who already has legitimate write access to the log file from editing it before the next verification run. Real-time prevention of that would require OS-level access control and continuous audit logging, which is outside this project's scope. Second, the design assumes a single trusted or at least singly accountable signing identity per custodian at any given time; if a private signing key is compromised, an attacker can forge subsequent entries that are indistinguishable from legitimate ones, a risk that multi-signature or HSM-backed key storage would mitigate but that this cycle did not implement. Third, and most fundamentally, the entire no-consensus argument stands or falls on the single-custodian assumption. If evidence ever needed to be jointly attested across multiple, mutually distrusting labs with no shared trust anchor between them, this design's assumptions would no longer hold, and a consensus mechanism would become the right tool for that different problem.

The folder-batch extension (Sections V-E, VI-E) is worth a brief separate note, since it changes the shape of the Merkle tree without changing its guarantees: nesting one tree inside another only works because the second hop's proof steps are computed over the same hash-linking primitive as the first hop, so verification composes cleanly without needing any new cryptographic assumption. The one real engineering cost this introduced was in the standalone verifier since it deliberately shares no code with the logging tool, the two-hop verification logic had to be reimplemented independently rather than reused, and that duplication is now a second place a future format change would need to be applied in lockstep. We consider that an acceptable, explicit

cost for keeping the verifier's independence, not an oversight.

VIII. CONCLUSION AND FUTURE WORK

This paper presented a chain-of-custody system for digital forensic evidence that achieves blockchain-grade tamper-evidence cryptographic hashing, hash-linked custody records, Merkle-tree inclusion proofs, and ECDSA-signed actor attribution without distributed consensus, on the grounds that a single-custodian forensic lab does not have the multi-party trust problem consensus exists to solve. An independently implemented standalone verifier, an encrypted-at-rest SQLite storage backend, a menu-driven interactive shell, folder-batch evidence registration with nested Merkle proofs, and a tamper-simulation harness demonstrating 100% detection across four corruption scenarios together support that claim.

Several extensions were considered but not selected this cycle, and remain as future work: RFC 3161 trusted timestamping or OpenTimestamps external anchoring, which would reinforce the integrity claims with a third-party time reference without contradicting the no-consensus thesis; multi-signature or threshold signing and HSM- or TPM-backed key storage, which would directly address the key-compromise boundary noted in Section VII; a post-quantum cryptography comparison between Dilithium and ECDSA; a FastAPI REST wrapper or web dashboard with role-based access control; support for additional evidence-container formats such as E01, dd, pcap, and mobile extraction formats; NSRL hash-set cross-referencing; court-ready report generation; QR-code evidence tagging; and a continuous-integration pipeline. On the experimental side, isolating SQLite's page-allocation overhead from AES-GCM's own ciphertext expansion by adding an unencrypted-SQLite baseline alongside the current plaintext-JSON baseline is a natural next refinement before this line of results is finalized for submission.

ACKNOWLEDGMENT

The authors thank Dr. Archana Kalidindi, Assistant Professor, Department of Computer Science and Engineering, KL University (Hyderabad), for supervision and guidance throughout this project.

IX. REFERENCES

References are listed in APA style. Reference 9 reflects the correction identified during this project's citation re-verification pass (Section III).

[1] Ahmad, L., Khanji, S., Iqbal, F., & Kamoun, F. (2020). Blockchain-based chain of custody: Towards real-time tamper-proof evidence management. In Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES '20). ACM.

[2] Al-Khateeb, H. M., Epiphaniou, G., & Daly, H. (2019). Blockchain for modern digital forensics: The chain-of-custody as a distributed ledger.

[3] Bonomi, S., Casini, M., & Ciccotelli, C. (2018). B-CoC: A blockchain-based chain of custody for evidences management in digital forensics. arXiv preprint. Sapienza Università di Roma.

[4] Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In Proceedings of the 18th USENIX Security Symposium.

[5] Lone, A. H., & Mir, R. N. (2019). Forensic-chain: Blockchain based digital forensics chain of custody with PoC in Hyperledger Composer. Digital Investigation.

[6] Miller, A., & Singh, A. (2024). Chain of custody and evidence integrity verification using blockchain technology. In Proceedings of the International Conference on Cyber Warfare and Security (ICCWS 2024). *[Corrected from "Smart Contracts" / 2021 as recorded in an earlier literature summary; verified against the official ICCWS proceedings listing.]*

[7] Patil, H. E., Kohli, R. K., Puri, S., & Puri, P. (2024). Potential applicability of blockchain technology in the maintenance of chain of custody in forensic casework. Egyptian Journal of Forensic Sciences, 14(1).

[8] Prakash, F., & Sadawarti, H. (2022). Blockchain-based chain of custody: A secure digital evidence framework for digital forensics investigation. Applied Innovative Research, 3(1–4).

[9] Sakshi, Malik, A., & Sharma, A. K. (2023). Blockchain-based digital chain of custody multimedia evidence preservation framework for Internet-of-Things. Journal of King Saud University – Computer and Information Sciences.

[10] Yagiz, M. A., Horasan, F., & Yurttakal, A. H. (2026). Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle-tree pipeline with adaptive chunking. arXiv preprint.